

Mettre en œuvre des solutions RAG

Prérequis

[Provision an Azure SQL Database](#)

[Créer un projet Foundry et déployez des modèles Azure OpenAI](#)

[Configurer l'accès à l'identité gérée](#)

[Créer le tableau ProductReview](#)

[Créer une base de données avec un champ d'identification et un modèle externe pour les embeddings](#)

[Ajouter une colonne vectorielle et générez des embeddings](#)

[Récupérer les données en utilisant la recherche vectorielle et les formater en contexte JSON](#)

[Construis une invite augmentée avec le contexte de la base de données](#)

[Appelez le point de terminaison Azure OpenAI et générez une réponse](#)

[Créer une procédure stockée RAG](#)

[Nettoyage](#)

Temps estimé : 45 minutes

Dans cet exercice, vous implémentez une solution complète de génération augmentée par récupération (RAG) en utilisant Azure SQL Database. Vous créez une table d'avis produit, ajoutez une colonne vectorielle pour stocker les embeddings, générez des embeddings pour les avis clients, utilisez la recherche vectorielle pour récupérer les avis pertinents, les formatez en contexte JSON, construisez une invite augmentée, appelez un endpoint Azure OpenAI, et extrayez la réponse.

Vous êtes développeur de bases de données pour Adventure Works. Votre équipe souhaite construire un assistant produit alimenté par l'IA qui répond aux questions des clients en utilisant de véritables avis clients. Au lieu d'affiner un modèle, vous utilisez RAG pour ancrer les réponses du modèle dans votre base de données. Vous utilisez la recherche vectorielle pour trouver les avis les plus pertinents pour chaque question.

! Ces exercices vous demandent de copier-coller du code T-SQL. Veuillez vérifier que le code a été copié correctement avant de l'exécuter.

Prérequis

- [Un abonnement Azure](#) avec approbation pour [l'accès Azure OpenAI](#)
- Visual Studio Code avec l'extension SQL Server (mssql), ou SQL Server Management Studio
- Connaissance de base de Azure SQL Database et T-SQL

Provision an Azure SQL Database

Tout d'abord, créez une base de données Azure SQL avec des données d'exemple.

! Passez cette section si vous avez déjà une base de données Azure SQL AdventureWorksLT provisionnée.

1. Allez sur le [hub Azure SQL](#) et connectez-vous avec votre compte Azure si vous le demandez. Dans le volet **Azure SQL Database**, sélectionnez **Afficher les options**, puis **Créer une base de données SQL**.

! Si vous voyez une bannière **d'offre gratuite** sur cette page, vous pouvez l'utiliser gratuitement pour utiliser Azure SQL Database. [L'offre gratuite](#) offre 100 000 vCore secondes de calcul serverless et 32 Go de stockage par mois. Si vous postulez pour l'offre gratuite, passez les étapes 3 à 6.

2. Dans l'onglet **Bases** de la page **Créer une base de données SQL**, remplissez les informations requises :

Cadre	Valeur
Abonnement	Sélectionnez votre abonnement Azure.
Groupe de ressources	Sélectionnez ou créez un groupe de ressources.
Nom de la base de données	<i>AdventureWorksLT</i>
Serveur	Sélectionnez Créer un nouveau serveur et créez un nouveau serveur avec un nom unique. Sélectionnez votre localisation . Pour l'authentification, sélectionnez l'une des options suivantes puis sélectionnez OK :



! L'authentification

n'est pas optionnelle.

Vous devez choisir la méthode qui correspond aux politiques de sécurité de votre organisation. Chaque option influence la façon dont vous vous connectez à la base de données ultérieurement :

- **Utilisez l'authentification Microsoft Entra uniquement (recommandée) :** Sélectionnez ceci si votre organisation nécessite un accès basé sur Entra. Définissez votre compte Azure comme **administrateur Microsoft Entra**. Vous vous connectez à la base de données en utilisant votre compte Microsoft Entra (par exemple, dans SSMS, sélectionnez *Authentification* = **Microsoft Entra MFA**).
- **Utilisez à la fois l'authentification SQL et Microsoft Entra/SQL :** Sélectionnez ceci si vous préférez une connexion administrateur SQL ou si votre organisation autorise les deux méthodes. Fournissez un **identifiant administrateur** serveur et un **mot de passe**. Vous avez besoin de ces identifiants pour vous connecter. Vous pouvez aussi configurer un **administrateur Microsoft Entra** pour activer les connexions Entra en plus de l'auth-licence SQL.

La méthode d'authentification que vous choisissez ici contrôle la façon *dont vous vous* connectez à la base de données en tant que développeur. C'est distinct de la façon dont Azure SQL se connecte à Azure OpenAI, qui est configuré plus tard dans le laboratoire.

! Si vous avez déjà un serveur de test que vous pouvez utiliser, sélectionnez-le au lieu d'en créer un nouveau.

3. Laisser **Vouloir utiliser SQL elastic pool** réglé sur **Non**.
4. Pour **l'environnement de charge de travail**, sélectionnez **Développement**. Cela préconfigure le calcul en **mode serveur à usage général** avec l'auto-pause activée, qui est l'option payante la plus économique.
5. Dans **Calculer + stockage**, sélectionnez **Configurer la base de données**. Changez la couche de service en **Hyperscale** et la couche de calcul en **Serverless**. **Sélectionnez Postuler** pour confirmer.
6. Dans la section **Redondance du stockage de sauvegarde**, sélectionnez **Stockage de sauvegarde localement redondant**.
7. Sélectionner **Suivant : Réseautage**.
8. Dans l'onglet **Réseau**, pour **la méthode de connectivité**, sélectionnez **Terminaison public**.

! Si vous avez sélectionné un serveur existant au lieu d'en créer un nouveau, l'option **méthode de connectivité** peut ne pas apparaître car elle est déjà configurée sur le serveur.

9. Dans **les règles du pare-feu**, configurez **Oui : Autoriser les services et ressources Azure à accéder à ce serveur et Ajouter l'adresse IP client actuelle à Oui**.
10. Select **Next: Security**, then select **Next: Additional settings**.
11. On the **Additional settings** tab, under **Data source**, set **Use existing data** to *Sample* to create the AdventureWorksLT sample database. Select **OK** when prompted to confirm.
12. Select **Review + create**, review the settings, and then select **Create**.
13. Wait for the deployment to complete, then navigate to the new Azure SQL Database resource.

Create a Foundry project and deploy Azure OpenAI models

Next, create a project in Microsoft Foundry and deploy a chat model and an embedding model.

! Skip this section if you already have an Azure OpenAI resource with **gpt-5.4-mini** and **text-embedding-3-small** models deployed.



Create a Foundry project

The first step is to create a new project in Microsoft Foundry, which will serve as the workspace for deploying and managing your Azure OpenAI models.

1. Go to [Microsoft Foundry](#) and sign in with your Azure account.

! If you see a **New Foundry** toggle in the upper-right corner of the portal, make sure it is turned **on** to use the latest version of the Foundry portal. The steps below assume the new experience.

2. Create a new project:

- a. If a project name is shown in the upper-left corner, select it and then select **Create new project**. If no project exists, select **+ Create project** from the home page.
- b. Enter a project name (for example, *proj-sqlailab*).
- c. Expand **Advanced options**. Select the same **Subscription** and **Resource group** you used for your Azure SQL Database, and choose a **Region** where Azure OpenAI models are available.
- d. Select **Create**. If a **Let's go** prompt appears, select **Let's go**.

Deploy Azure OpenAI models

You will deploy two models: **gpt-5.4-mini** (for chat completions) and **text-embedding-3-small** (for embeddings). The following steps guide you through deploying both models.

Let's deploy the **gpt-5.4-mini** model first.

1. Select **Discover** in the top navigation bar. In the search bar at the top of the page, search for **gpt-5.4-mini**. Alternatively, select **View all models** in the **Featured models** section to browse the full catalog. Select **gpt-5.4-mini** from the search results.
2. On the model details page, select the **Deploy** dropdown and choose **Default settings**.
3. If a **Select a project to deploy** dialog appears, select the same **Region** you chose when creating the project, then select your project from the **Project** dropdown. Select **Continue**.
4. Set the **Deployment name** to **gpt-5.4-mini** and select **Deploy**.
5. After deployment is complete, the model is now deployed.

Let's deploy the **text-embedding-3-small** model next.

1. Now deploy the embedding model. Select **Discover** in the top navigation and search for **text-embedding-3-small**. Select it from the search results.
2. On the model details page, select the **Deploy** dropdown and choose **Default settings**.
3. If a **Select a project to deploy** dialog appears, select the same **Region** and **Project** as before and select **Continue**.
4. After deployment is complete, the model is now deployed.
5. Select **Build** > **Models** in the top navigation to verify both deployments appear.

Retrieve the Azure OpenAI endpoint

Now retrieve the **Azure OpenAI endpoint** you need for the T-SQL steps later.

1. In the [Azure portal](#), navigate to your resource group and select the **Foundry** resource that was created with your project (for example, *proj-sqlailab-resource*).
2. In the left menu, select **Resource Management** > **Keys and Endpoint**. Select the **OpenAI** tab and note the endpoint URL (for example,). You need the endpoint name (the part before) later.

`https://proj-sqlailab-resource.openai.azure.com/` `.openai.azure.com`



! The **Keys and Endpoint** page has three tabs: **Foundry**, **OpenAI**, and **AI Services**. Make sure you select the **OpenAI** tab. This tab shows the endpoint in the format required by Azure SQL Database's and . The Foundry tab shows a different URL that does not work with these T-SQL features: `CREATE EXTERNAL MODEL`

```
sp_invoke_external_rest_endpoint .services.ai.azure.com
```

Configure managed identity access

Since Azure SQL Database uses a system-assigned managed identity to authenticate with Azure OpenAI, you need to enable the identity on your SQL Server and grant it access to the Azure OpenAI resource. This approach is more secure than API keys and doesn't require storing secrets.

! Skip this section if your SQL Server already has a system-assigned managed identity enabled and it has been granted the **Cognitive Services OpenAI User** role on your Azure OpenAI resource.

1. In the Azure portal, navigate to the **SQL server** you created earlier (the logical server, not the database).
2. In the left menu, select **Security > Identity**.
3. Under **System assigned managed identity**, set **Status** to **On** and select **Save**.
4. Once the managed identity is enabled on your SQL Server, navigate to your **Azure OpenAI** resource (for example, *adventureworks-openai*).
5. In the left menu, select **Access control (IAM)**.
6. Select **+ Add** and then select **Add role assignment**.
7. On the **Role** tab, search for and select **Cognitive Services OpenAI User**, then select **Next**.
8. On the **Members** tab, select **Managed identity**, then select **+ Select members**.
9. In the **Select managed identities** pane, set **Managed identity** to **SQL server**, select your SQL server from the list, and then select **Select**.
10. Select **Review + assign** twice to complete the role assignment.

! The role assignment may take up to 5 minutes to take effect. You can proceed with the next steps while waiting.

Create the ProductReview table

The AdventureWorksLT sample database contains product information but no customer reviews. In this step, you download and run a script that creates a **ProductReview** table with 140 realistic reviews across many product categories. These reviews give the RAG solution rich, varied content to search through.

1. Connect to your Azure SQL Database using Visual Studio Code (with the SQL Server extension) or SQL Server Management Studio.

! **How to connect** depends on which authentication method your organization supports and was configured during server creation:

- o **Microsoft Entra authentication:** In SSMS, set *Authentication* to **Microsoft Entra MFA** and sign in with your Azure account. In VS Code, select the **Microsoft Entra ID** authentication type when creating a connection profile.
- o **SQL authentication:** In SSMS or VS Code, enter the **Server admin login** and **Password** you specified during server creation, with *Authentication* set to **SQL Login**.

In both cases, set the **Server name** to and the **Database** to **AdventureWorksLT**.

```
<your-server-name>.database.windows.net
```

2. Download the review script from [product-reviews-insert.sql](#) and save it locally.
3. Open the downloaded file and run the entire script against your **AdventureWorksLT** database.



4. Verify the table was created and populated by running:

Sql 

```
SELECT COUNT(*) AS TotalReviews FROM dbo.ProductReview;
GO
```

  You should see **140** rows. The reviews cover bikes, tires, lights, helmets, gloves, maintenance tools, and more, with ratings ranging from 1 to 5 stars.

Create a database scoped credential and an external model for embeddings

Set up the credential and model reference needed to call Azure OpenAI from T-SQL. You create a single credential using the system-assigned managed identity of your Azure SQL Server. This credential is used by both (for embeddings) and (for chat completions), and eliminates the need for API keys. `CREATE EXTERNAL MODEL`

`sp_invoke_external_rest_endpoint`

  Skip this section if you already have a database scoped credential and an external embedding model configured for your Azure OpenAI endpoint.

1. to create a database scoped credential using managed identity, open a new query window and run the following script. Replace with the endpoint name you noted from the **OpenAI** tab (for example, if your endpoint is , use *proj-sqlailab-resource*). `<your-openai-endpoint>`

`https://proj-sqlailab-resource.openai.azure.com/`

Sql 

```
-- Create a database master key if one doesn't exist
IF NOT EXISTS (SELECT * FROM sys.symmetric_keys WHERE name = '##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<your strong password here>';
GO

-- Create a credential using Managed Identity
CREATE DATABASE SCOPED CREDENTIAL [https://<your-openai-endpoint>.openai.azure.com]
WITH IDENTITY = 'Managed Identity', SECRET =
'{"resourceid":"https://cognitiveservices.azure.com"}';
GO
```

  Replace with a strong password. Replace with the endpoint name from the **OpenAI** tab (for example, *proj-sqlailab-resource*).

`<your strong password here>`

`<your-openai-endpoint>`

 **Important:** Do **not** change the value in the SECRET. It must remain exactly . This is the fixed OAuth audience for Azure Cognitive Services, not your specific endpoint URL. Changing it will cause authentication errors. `resourceid`

`https://cognitiveservices.azure.com`

2. Now create an external model reference for the embedding model. This reference allows you to use directly in T-SQL. Replace with your endpoint name. `AI_GENERATE_EMBEDDINGS` `<your-openai-endpoint>`

Sql 



```
-- Create an external model reference for embeddings
CREATE EXTERNAL MODEL my_embedding_model
WITH (
    LOCATION = 'https://<your-openai-endpoint>.openai.azure.com/openai/deployments/text-embedding-3-small/embeddings?api-version=2024-10-21',
    API_FORMAT = 'Azure OpenAI',
    MODEL_TYPE = EMBEDDINGS,
    MODEL = 'text-embedding-3-small',
    CREDENTIAL = [https://<your-openai-endpoint>.openai.azure.com]
);
GO
```

!  Replace with the same endpoint name you used in the credential. The value is the current GA version of the Azure OpenAI REST API. The option is required and must match the model name. `<your-openai-endpoint>`

`api-version` `2024-10-21` `MODEL`

Add a vector column and generate embeddings

In this section, you add a vector column to the ProductReview table, generate embeddings for review text, and create a vector index for efficient similarity search.

!  Skip this section if your table already has a column with generated embeddings. `dbo.ProductReview`
`ReviewVector`

1. First, add a vector column to the table to store review embeddings: `dbo.ProductReview`

Sql  Copy

```
-- Add a vector column to store review text embeddings
ALTER TABLE dbo.ProductReview
ADD ReviewVector VECTOR(1536);
GO
```

!  The data type stores a 1536-dimensional vector, which matches the output of the *text-embedding-3-small* model. `VECTOR(1536)`

2. Generate embeddings for each review by combining the product name, review title, and review text. The script processes reviews in batches of 30 with a short delay between batches to avoid API rate limits:

Sql  Copy



```
-- Generate embeddings in batches to avoid API rate limits
DECLARE @batchSize INT = 30;
DECLARE @rowsUpdated INT = 1;
DECLARE @retryCount INT;
DECLARE @maxRetries INT = 3;

WHILE @rowsUpdated > 0
BEGIN
    SET @retryCount = 0;

    RETRY:
    BEGIN TRY
        UPDATE TOP (@batchSize) r
        SET r.ReviewVector = AI_GENERATE_EMBEDDINGS(
            p.Name + ' - ' + r.ReviewTitle + ': ' + r.ReviewText
            USE MODEL my_embedding_model)
        FROM dbo.ProductReview r
        INNER JOIN SalesLT.Product p ON r.ProductID = p.ProductID
        WHERE r.ReviewVector IS NULL;

        SET @rowsUpdated = @@ROWCOUNT;

        -- Brief pause between batches to respect API rate limits
        IF @rowsUpdated > 0
            WAITFOR DELAY '00:00:02';
    END TRY
    BEGIN CATCH
        SET @retryCount += 1;
        IF @retryCount <= @maxRetries
        BEGIN
            PRINT 'Rate limited. Retrying in 5 seconds... (Attempt '
                + CAST(@retryCount AS NVARCHAR(10)) + ' of '
                + CAST(@maxRetries AS NVARCHAR(10)) + ')';
            WAITFOR DELAY '00:00:05';
            GOTO RETRY;
        END
        ELSE
            THROW;
    END CATCH
END
GO
```

!  Cette étape peut prendre quelques minutes. Le script traite 30 revues par lot avec une pause de 2 secondes entre les lots. Si l'API renvoie une erreur de limite de vitesse, elle réessaie jusqu'à trois fois avec une attente de 5 secondes. Le nom du produit, le titre de l'avis et le texte de l'avis sont intégrés ensemble afin que la recherche vectorielle puisse correspondre à la fois au produit évalué et à l'expérience du client.

3. Vérifiez que les embeddings ont été générés :

SQL

 Bien reçu



```
-- Check how many reviews have embeddings
SELECT
    COUNT(*) AS TotalReviews,
    COUNT(ReviewVector) AS ReviewsWithEmbeddings
FROM dbo.ProductReview;
GO
```

4. Activez les fonctionnalités d'aperçu et créez un index vectoriel sur la colonne pour une recherche efficace du plus proche voisin approximatif (ANN) :

SQL

Bien reçu

```
-- Enable preview features required for vector indexes
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;
GO

-- Allow the table to remain writable after vector index creation
ALTER DATABASE SCOPED CONFIGURATION SET ALLOW_STALE_VECTOR_INDEX = ON;
GO

-- Create a DiskANN vector index for fast approximate nearest neighbor search
CREATE VECTOR INDEX IX_Review_ReviewVector
ON dbo.ProductReview(ReviewVector)
WITH (METRIC = 'cosine', TYPE = 'DISKANN');
GO
```

creates a DiskANN-based approximate nearest neighbor (ANN) index, which is fundamentally different from a regular nonclustered index. DiskANN builds a graph structure that navigates through vectors to find close matches efficiently. The setting keeps the table writable. Without it, the table becomes read-only when a vector index exists. For large tables, the ANN index dramatically speeds up similarity search by avoiding a full scan of every row.

`CREATE VECTOR INDEX` `ALLOW_STALE_VECTOR_INDEX`

Retrieve data using vector search and format it as JSON context

In this section, you practice the **retrieval** step of RAG. Instead of using a hardcoded clause, you convert the user's question into an embedding and use to find the most relevant reviews. Then you format the results as JSON. `WHERE` `VECTOR_SEARCH`

1. Run the following query to use the function for approximate nearest neighbor search. This function uses the DiskANN vector index for fast retrieval: `VECTOR_SEARCH`

Sql

Copy



```
-- Convert a question to an embedding and find the closest matching reviews
DECLARE @userQuestion NVARCHAR(1000) = 'What mountain bike can handle really technical
rocky trails?';
DECLARE @questionVector VECTOR(1536);

-- Generate embedding for the question
SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@userQuestion USE MODEL
my_embedding_model);

-- Find the top 5 most relevant reviews using ANN vector search
SELECT
    p.Name AS ProductName,
    p.ListPrice,
    pc.Name AS Category,
    r.Rating,
    r.ReviewTitle,
    r.ReviewText,
    vs.distance AS Distance
FROM VECTOR_SEARCH(
    TABLE = dbo.ProductReview AS r,
    COLUMN = ReviewVector,
    SIMILAR_TO = @questionVector,
    METRIC = 'cosine',
    TOP_N = 5
) AS vs
INNER JOIN SalesLT.Product p
    ON r.ProductID = p.ProductID
INNER JOIN SalesLT.ProductCategory pc
    ON p.ProductCategoryID = pc.ProductCategoryID
FOR JSON PATH;
GO
```

!  performs an approximate nearest neighbor (ANN) search using the DiskANN vector index. Unlike with (which scans every row), navigates the graph index to find the closest matches efficiently. The column is automatically included in the results. Lower distance values mean higher relevance. Notice how each review is unique. Unlike product descriptions where multiple sizes of the same bike share the same text, each review describes a distinct customer experience. `VECTOR_SEARCH` `VECTOR_DISTANCE` `ORDER BY` `VECTOR_SEARCH` `distance`

Build an augmented prompt with database context

Now you combine retrieved data with a system message and user question to build the **augmented** prompt. This prompt is the “A” in RAG.

1. Run the following script to build a complete RAG prompt in T-SQL. This script uses vector search to retrieve relevant reviews, then builds the augmented prompt:

Sql

 Copy



```
DECLARE @userQuestion NVARCHAR(1000) = 'What is the best bike for commuting to work in
rainy weather?';
DECLARE @questionVector VECTOR(1536);
DECLARE @context NVARCHAR(MAX);
DECLARE @payload NVARCHAR(MAX);

-- Step 1: Convert the question to an embedding
SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@userQuestion USE MODEL
my_embedding_model);

-- Step 2: Retrieve relevant reviews using ANN vector search
SET @context = (
    SELECT
        p.Name AS ProductName,
        p.ListPrice,
        pc.Name AS Category,
        r.Rating,
        r.ReviewTitle,
        r.ReviewText
    FROM VECTOR_SEARCH(
        TABLE = dbo.ProductReview AS r,
        COLUMN = ReviewVector,
        SIMILAR_TO = @questionVector,
        METRIC = 'cosine',
        TOP_N = 5
    ) AS vs
    INNER JOIN SalesLT.Product p
        ON r.ProductID = p.ProductID
    INNER JOIN SalesLT.ProductCategory pc
        ON p.ProductCategoryID = pc.ProductCategoryID
    FOR JSON PATH
);

-- Step 3: Build augmented prompt using JSON_OBJECT and JSON_ARRAY
SET @payload = JSON_OBJECT(
    'messages': JSON_ARRAY(
        JSON_OBJECT(
            'role': 'system',
            'content': 'You are an Adventure Works product assistant. Answer questions
using only the provided product reviews and data. Mention specific customer experiences from
the reviews when relevant. Be concise and helpful. If the data does not contain enough
information, say so.'
        ),
        JSON_OBJECT(
            'role': 'user',
            'content': 'Product reviews: ' + ISNULL(@context, '[]') + CHAR(10) + CHAR(10) +
'Customer question: ' + @userQuestion
        )
    ),
    'max_tokens': CAST(500 AS INT),
    'temperature': 0.5
);

-- Display the constructed payload
SELECT @payload AS AugmentedPrompt;
GO
```



!  Review the output JSON. The function finds the most semantically relevant customer reviews using the DiskANN vector index. Each review is unique, containing a real customer's experience, rating, and opinion, which gives the model richer context than product descriptions alone. The system message instructs the model to reference specific customer experiences in its answers. `VECTOR_SEARCH`

Call the Azure OpenAI endpoint and generate a response

This step is the "G" in RAG, the generation step. You send the augmented prompt to Azure OpenAI and extract the answer.

1. Run the following script to complete the full RAG pipeline. Replace with your endpoint name.

```
<your-openai-endpoint>
```

```
Sql
```

 Copy



```
DECLARE @userQuestion NVARCHAR(1000) = 'Which tires last the longest and resist
punctures?';
DECLARE @questionVector VECTOR(1536);
DECLARE @context NVARCHAR(MAX);
DECLARE @payload NVARCHAR(MAX);
DECLARE @response NVARCHAR(MAX);
DECLARE @returnValue INT;

-- Step 1: Convert the question to an embedding
SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@userQuestion USE MODEL
my_embedding_model);

-- Step 2: Retrieve relevant reviews using ANN vector search
SET @context = (
    SELECT
        p.Name AS ProductName,
        p.ListPrice,
        pc.Name AS Category,
        r.Rating,
        r.ReviewTitle,
        r.ReviewText
    FROM VECTOR_SEARCH(
        TABLE = dbo.ProductReview AS r,
        COLUMN = ReviewVector,
        SIMILAR_TO = @questionVector,
        METRIC = 'cosine',
        TOP_N = 5
    ) AS vs
    INNER JOIN SalesLT.Product p
        ON r.ProductID = p.ProductID
    INNER JOIN SalesLT.ProductCategory pc
        ON p.ProductCategoryID = pc.ProductCategoryID
    FOR JSON PATH
);

-- Step 3: Build augmented prompt
SET @payload = JSON_OBJECT(
    'messages': JSON_ARRAY(
        JSON_OBJECT(
            'role': 'system',
            'content': 'You are an Adventure Works product assistant. Answer questions
using only the provided product reviews and data. Mention specific customer experiences from
the reviews when relevant. Be concise and helpful. If the data does not contain enough
information, say so.'
        ),
        JSON_OBJECT(
            'role': 'user',
            'content': 'Product reviews: ' + ISNULL(@context, '[]') + CHAR(10) + CHAR(10) +
'Customer question: ' + @userQuestion
        )
    ),
    'max_tokens': CAST(500 AS INT),
    'temperature': 0.5
);

-- Step 4: Call Azure OpenAI
EXECUTE @returnValue = sp_invoke_external_rest_endpoint
    @url = N'https://<your-openai-endpoint>.openai.azure.com/openai/deployments/gpt-5.4-
```



```
mini/chat/completions?api-version=2024-10-21',
    @method = 'POST',
    @payload = @payload,
    @credential = [https://<your-openai-endpoint>.openai.azure.com],
    @response = @response OUTPUT;

-- Step 5: Extract and display the answer
IF @returnValue = 0
BEGIN
    DECLARE @answer NVARCHAR(MAX);
    SET @answer = JSON_VALUE(@response, '$.result.choices[0].message.content');
    SELECT @answer AS AssistantResponse;
END
ELSE
BEGIN
    SELECT
        @returnValue AS HttpStatus,
        JSON_VALUE(@response, '$.response.status.http.description') AS ErrorDescription;
END
GO
```

!  Replace with the same endpoint name used in your credential. The value is the current GA version of the Azure OpenAI REST API.

Create a RAG stored procedure

Now put it all together in a reusable stored procedure that your application can call.

1. Run the following script to create the stored procedure. Replace with your endpoint name.

Sql

 Copy



```
CREATE OR ALTER PROCEDURE dbo.AskProductQuestion
    @Question NVARCHAR(1000),
    @Answer NVARCHAR(MAX) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @questionVector VECTOR(1536);
    DECLARE @context NVARCHAR(MAX);
    DECLARE @payload NVARCHAR(MAX);
    DECLARE @response NVARCHAR(MAX);
    DECLARE @returnValue INT;

    -- Step 1: Convert the question to an embedding
    SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@Question USE MODEL
my_embedding_model);

    -- Step 2: Retrieve relevant reviews using ANN vector search
    SET @context = (
        SELECT
            p.Name AS ProductName,
            p.ListPrice,
            pc.Name AS Category,
            r.Rating,
            r.ReviewTitle,
            r.ReviewText
        FROM VECTOR_SEARCH(
            TABLE = dbo.ProductReview AS r,
            COLUMN = ReviewVector,
            SIMILAR_TO = @questionVector,
            METRIC = 'cosine',
            TOP_N = 5
        ) AS vs
        INNER JOIN SalesLT.Product p
            ON r.ProductID = p.ProductID
        INNER JOIN SalesLT.ProductCategory pc
            ON p.ProductCategoryID = pc.ProductCategoryID
        FOR JSON PATH
    );

    -- Check if context was retrieved
    IF @context IS NULL
    BEGIN
        SET @Answer = 'No reviews found matching your query. Please try a different
question.';
        RETURN;
    END

    -- Step 3: Build the augmented prompt
    SET @payload = JSON_OBJECT(
        'messages': JSON_ARRAY(
            JSON_OBJECT(
                'role': 'system',
                'content': 'You are an Adventure Works product assistant. Follow these
rules:
1. Answer only using the provided product reviews and data
2. Reference specific customer experiences from the reviews when relevant
3. Include star ratings to help the customer assess product quality
```



```
4. Keep responses under 150 words
5. Suggest related products when relevant'
    ),
    JSON_OBJECT(
        'role': 'user',
        'content': 'Product reviews: ' + @context + CHAR(10) + CHAR(10) + 'Customer
question: ' + @Question
    )
),
'max_tokens': CAST(500 AS INT),
'temperature': 0.5
);

-- Step 4: Call the model
EXECUTE @returnValue = sp_invoke_external_rest_endpoint
    @url = N'https://<your-openai-endpoint>.openai.azure.com/openai/deployments/gpt-
5.4-mini/chat/completions?api-version=2024-10-21',
    @method = 'POST',
    @payload = @payload,
    @credential = [https://<your-openai-endpoint>.openai.azure.com],
    @response = @response OUTPUT;

-- Step 5: Extract the answer or handle errors
IF @returnValue = 0
    SET @Answer = JSON_VALUE(@response, '$.result.choices[0].message.content');
ELSE IF @returnValue = 429
    SET @Answer = 'The service is currently busy. Please try again in a moment.';
ELSE IF @returnValue IN (401, 403)
    SET @Answer = 'Authentication failed. Please check the credential configuration.';
ELSE
    SET @Answer = 'Unable to process your question at this time. HTTP status: ' +
CAST(@returnValue AS NVARCHAR(10));
END;
GO
```

!  Replace with the same endpoint name used in your credential. The value is the current GA version of the Azure OpenAI REST API. `<your-openai-endpoint>` `api-version` `2024-10-21`

2. Test the stored procedure with a question about night riding safety:

Sql	 Copy
<pre>DECLARE @response NVARCHAR(MAX); EXEC dbo.AskProductQuestion @Question = 'I ride before sunrise and after dark. What lights actually work well?', @Answer = @response OUTPUT; SELECT @response AS AssistantResponse; GO</pre>	

3. Try a question about bike maintenance:

Sql	 Copy
-----	--



```
DECLARE @response NVARCHAR(MAX);

EXEC dbo.AskProductQuestion
    @Question = 'How do I keep my bike maintained between shop visits?',
    @Answer = @response OUTPUT;

SELECT @response AS AssistantResponse;
GO
```

4. Test with a question about common complaints:

Sql Copy

```
DECLARE @response NVARCHAR(MAX);

EXEC dbo.AskProductQuestion
    @Question = 'What are the most common problems or complaints people have with their bikes?',
    @Answer = @response OUTPUT;

SELECT @response AS AssistantResponse;
GO
```

!  Each call follows the same RAG pattern: convert the question to an embedding, use with the DiskANN index to retrieve the most relevant reviews from the database, augment the prompt with that data, and generate a grounded response. The model's answers should reference specific customer experiences from the reviews. Notice how vector search finds relevant reviews based on semantic meaning. For example, a question about "problems" retrieves low-rated reviews mentioning specific issues. **VECTOR_SEARCH**

Try your own questions as well! The more you test with different queries, the better you understand how the RAG pipeline works and how the model uses the retrieved context to generate answers.

Cleanup

If you aren't using the Azure SQL Database or the Azure OpenAI resources for any other purpose, you can clean up the resources you created in this exercise.

!  These resources are used on labs 9, 10, and 11.

Si vous avez provisionné un nouveau groupe de ressources pour ce laboratoire, vous pouvez simplement supprimer tout le groupe de ressources pour supprimer toutes les ressources d'un coup. Si vous avez utilisé un groupe de ressources existant, supprimez individuellement la base de données Azure SQL et la ressource Azure OpenAI.

1. Dans le portail Azure, allez dans votre groupe de ressources.
2. Sélectionnez **Supprimer le groupe de ressources** et confirmez la suppression en tapant le nom du groupe de ressources.
3. **Sélectionnez Supprimer** pour supprimer toutes les ressources créées dans ce laboratoire.

Vous avez réussi cet exercice.

Dans cet exercice, vous avez implémenté une solution complète de génération augmentée par la récupération (RAG) en utilisant Azure SQL Database et Azure OpenAI. Vous avez généré des embeddings et créé un index vectoriel pour une récupération rapide, recherché des revues sémantiquement pertinentes, formaté les résultats



comme contexte pour un modèle de langage, construit des invites augmentées avec instructions de mise à la terre, appelé le point de terminaison Azure OpenAI de T-SQL, et empaqueté tout le pipeline RAG dans une procédure stockée réutilisable avec gestion des erreurs.